



HEAVY WIZARDRY 101

Shellcodes, Backdoors, Droppers, and Worms

by David Martínez Oliveira, aka Pico



**no starch
press®**

San Francisco

HEAVY WIZARDRY 101. Copyright © 2026 by David Martínez Oliveira, aka Pico.

All rights reserved. No part of this work may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or by any information storage or retrieval system, without the prior written permission of the copyright owner and the publisher.

First printing

30 29 28 27 26 1 2 3 4 5

ISBN-13: 978-1-7185-0442-4 (print)

ISBN-13: 978-1-7185-0443-1 (ebook)



Published by No Starch Press®, Inc.
245 8th Street, San Francisco, CA 94103
phone: +1.415.863.9900
www.nostarch.com; info@nostarch.com

Publisher: William Pollock
Managing Editor: Jill Franklin
Production Manager: Sabrina Plomitallo-González
Production Editor: Sydney Cromwell
Developmental Editor: Nathan Heidelberger
Cover Illustrator: Rob Fiore
Interior Design: Octopod Studios
Technical Reviewer: Sy Brand
Copyeditor: Rachel Wheeler
Proofreader: Céline Parent
Library of Congress Control Number: 2025038169

For customer service inquiries, please contact info@nostarch.com. For information on distribution, bulk sales, corporate sales, or translations: sales@nostarch.com. For permission to translate this work: rights@nostarch.com. To report counterfeit copies or piracy: counterfeit@nostarch.com. The authorized representative in the EU for product safety and compliance is EU Compliance Partner, Pärnu mnt. 139b-14, 11317 Tallinn, Estonia, hello@eucompliancepartner.com, +3375690241.

No Starch Press and the No Starch Press iron logo are registered trademarks of No Starch Press, Inc. Other product and company names mentioned herein may be the trademarks of their respective owners. Rather than use a trademark symbol with every occurrence of a trademarked name, we are using the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The information in this book is distributed on an “As Is” basis, without warranty. While every precaution has been taken in the preparation of this work, neither the author nor No Starch Press, Inc. shall have any liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the information contained in it.

About the Author

David Martínez Oliveira, aka Pico, is a telecommunications engineer with over 25 years of experience developing low-level software for computer vision, communications, laboratory equipment, and AR/VR/MR applications. He moved to GNU/Linux in the ‘90s and never looked back. In his free time, he studies security, electronics, and hacking, and then shares what he learns.

About the Technical Reviewer

Sy Brand works on WebAssembly tooling at Fastly. With more than 10 years’ experience in developer tooling, they’ve worked on profilers, compilers, language runtimes, standard libraries, and debuggers. They’ve also contributed to the standards for C++, DWARF, and HSA. Brand is the author of *Building a Debugger* (No Starch Press, 2025).

BRIEF CONTENTS

Acknowledgments

Introduction

PART I: INTO THE MACHINE: AWAKENING CODE SPELLS

Chapter 1: Spell of the Binary Oracle: When Hardware Whispers in Opcodes

Chapter 2: Spell of Essence: Distilling C to Its Primordial ASM

Chapter 3: Spell of Rebirth: Transmuting Greetings into Shellcode

Chapter 4: Spell of the Overflowing Frame: Mastering Stack Secrets

Chapter 5: Spell of the Undead: Writing a REPL to Animate a Botnet

PART II: ONTO THE NETWORK: UNVEILING MYSTIC GATEWAYS

Chapter 6: Veil of Shadows: Unlocking Hidden Backdoors

Chapter 7: Veil of Ash: The Dropper That Leaves No Flame Behind

PART III: THROUGH THE NETWORK: CASTING CRAWLERS

Chapter 8: Casting the Twin Serpent: Summoning a Two-Headed Worm

Chapter 9: Casting the Lone Strand: Single-Stage Wormcraft

Chapter 10: Casting the Lost Hosts: Conjuring Network Scanners

APPENDIXES: THE FORGE

Appendix A: The Sorcerer’s Forge: Preparing Your Workshop

Appendix B: Secrets of the Forge: Arcane Tricks for Low-Level Tools

Appendix C: The Sorcerer’s Anvil: Full Spellcraft Listings

CONTENTS IN DETAIL

ACKNOWLEDGMENTS

INTRODUCTION

About This Book
Who This Book Is For
The Approach
What You'll Need
What's in the Book

PART I

INTO THE MACHINE: AWAKENING CODE SPELLS

1

SPELL OF THE BINARY ORACLE: WHEN HARDWARE WHISPERS IN OPCODES

A Computer as a Stack
 From the Bottom Up
 From the Top Down
The Simplest Computer in the World
 The Memory
 The CPU
 A Complete Computer Model
How Programs Work
 Writing Your First Program
 Running Your First Program
Your First C Program
Assembling Your First Program for Different Processors
 ARM
 MIPS
 RISC-V
Summary

2

SPELL OF ESSENCE: DISTILLING C TO ITS PRIMORDIAL ASM

Your First Real ASM Program

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

- Producing Your First Binary
 - Compiling and Linking the Program
- Interacting with the OS Through System Calls
- From C Down to ASM
 - Dissecting the C Code
 - Using `_start` vs. `main`
 - Generating Static vs. Dynamic Binaries
 - Getting Rid of `libc`
 - Avoiding an Executable Stack
- Aren't Those Binaries Too Big?
- A Closer Look at the Linker
 - Customizing Default Values with Linker Flags
 - Compiling and Linking C Programs Separately
 - Building ASM Programs Using `libc`
 - Handling Standard Libraries and Start Files
- System Call Conventions Across Architectures
 - x86_64
 - ARM32
 - AArch64
 - MIPS32
 - MIPS64
 - RISC-V

Summary

3

SPELL OF REBIRTH: TRANSMUTING GREETINGS INTO SHELLCODE

- Representing Information
 - Bits, Bytes, Words, and Beyond
 - Processor Native Word Sizes
 - Decimal, Binary, Hexadecimal, and More
 - Negative Numbers
 - Text Strings
- A Closer Look at Memory
 - A Simplified Hardware Memory Model
 - Native Word Size and Memory Alignment
 - Little vs. Big Endian
- “Hello, World!” in ASM
 - Labels and Assembler Commands
 - Pointers
- “Hello, World!” in C
 - Using the GNU Debugger
 - Declaring Pointers in C
- Writing Your First Shellcode
 - x86_64

- ARM32
- A Short Digression on Addressing Modes
- AArch64
- MIPS32
- MIPS64
- RISC-V

Summary

4

SPELL OF THE OVERFLOWING FRAME: MASTERING STACK SECRETS

An Overview of Stacks

- How Stacks Are Implemented
- How Stacks Are Used

The Basics of Functions

- Function Support in Assembly
- Parameters and Return Values

The Stack Frame

- Function Prologues and Epilogues
- Do We Need the Stack Frame?
- What Else Happens When You Leave a Function?
- Local Variables
- Declaration vs. Initialization
- Function Parameters

Buffer Overflows

- Creating a Vulnerable Program
- Instrumenting the Binary
- Triggering an Overflow
- Writing Your First Exploit

ARM32 Functions

- Simple Local Variables
- Local Buffers and Canaries

AArch64 Functions

- Simple Local Variables
- Local Buffers and Canaries

MIPS Functions

- Simple Local Variables
- Local Buffers and Canaries

RISC-V Functions

- Simple Local Variables
- Local Buffers and Canaries

Summary

5

SPELL OF THE UNDEAD: WRITING A REPL TO ANIMATE A BOTNET

A REPL in C

- Arrays
- Loops
- Conditionals

A REPL for x86_64

- Your First Library
- The REPL
- Jump Instructions
- Processor Flags
- Buffer Initialization
- String Instructions

A REPL for ARM

- Jump Instructions
- Flags and Conditions
- Buffer Initialization

A REPL for AArch64

- Jump Instructions
- Buffer Initialization

A REPL for MIPS

- Jump Instructions
- Buffer Initialization

A REPL for RISC-V

- Jump Instructions
- Buffer Initialization

Creating a Minimal Botnet

- The C2 REPL
- The Zombie REPL

This document was truncated here because it was created in the Evaluation Mode.

ASPOSE
Your File Format APIs